Input Output

Main Points

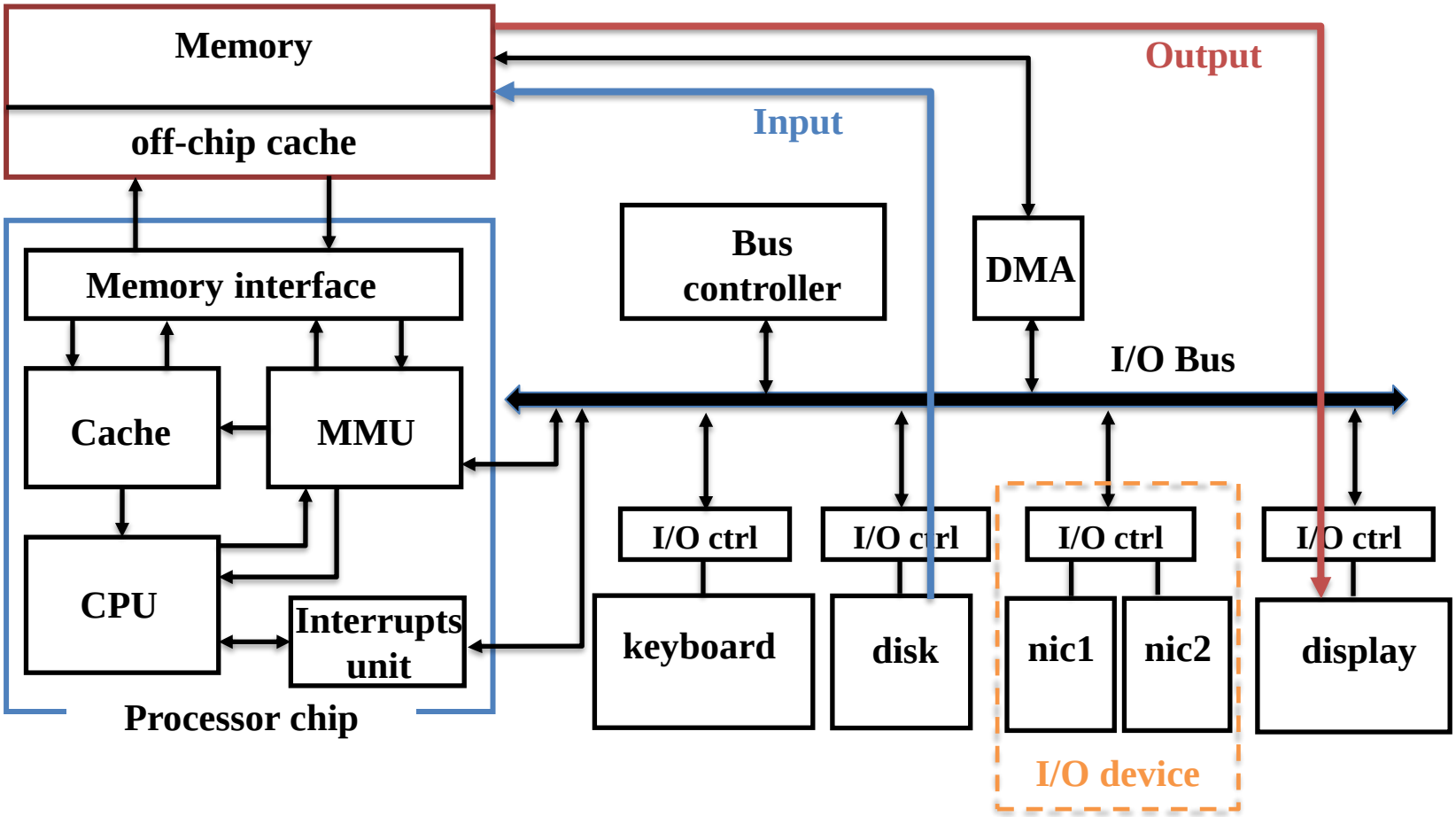
- I/O devices
- Device controller
- Buses
- I/O management
- Device drivers

Credits: some figures in these slides come from “Computer Organization and Architecture” by William Stallings, 10th edition.

Some I/O devices

Device	Behavior	Partner	Data rate (Mbit/sec)
Keyboard	Input	Human	0.0001
Mouse	Input	Human	0.0038
Voice input	Input	Human	0.2640
Sound input	Input	Machine	3.0000
Scanner	Input	Human	3.2000
Voice output	Output	Human	0.2640
Sound output	Output	Human	8.0000
Laser printer	Output	Human	3.2000
Graphics display	Output	Human	800.0000–8000.0000
Cable modem	Input or output	Machine	0.1280–6.0000
Network/LAN	Input or output	Machine	100.0000–10000.0000
Network/wireless LAN	Input or output	Machine	11.0000–54.0000
Optical disk	Storage	Machine	80.0000–220.0000
Magnetic tape	Storage	Machine	5.0000–120.0000
Flash memory	Storage	Machine	32.0000–200.0000
Magnetic disk	Storage	Machine	800.0000–3000.0000

Input/Output and I/O devices



I/O impact

- The I/O impact on program execution time may be quite high
- Let's suppose $T_{exe} = T_{cpu} + T_{I/O}$ and $T_{I/O} = \frac{1}{10} T_{cpu}$, therefore $T_{exe} = \frac{11}{10} T_{cpu}$
- If we speed up T_{cpu} by 10 times leaving unaltered $T_{I/O}$, we have $T_{cpu}^{enhanced} = \frac{1}{10} T_{cpu}$, thus $T_{exe} = \frac{1}{10} T_{cpu} + \frac{1}{10} T_{cpu} = \frac{1}{5} T_{cpu}$
- Let's consider the $Speedup = \frac{Exec.time\ before\ enhancement}{Exec.time\ after\ enhancement}$
- The speedup obtained is $\frac{11}{2} = 5.5$

An optimization of 10-fold on the T_{cpu} produced only about 5-fold enhancement on T_{exe}! Why?

Amdahl's law

- Proposed by Gene Amdahl in 1967. It deals with the potential maximum speedup attainable by a parallel program when increasing the number of processors from 1 to *N*
  - *It can be applied to any optimization process*
- Consider a program in which only the fraction *f* can be optimized (e.g., parallelized using N processors), whereas the fraction *(1-f)* remains unaltered (e.g., it is inherently sequential).
- $Speedup = \frac{Exec.time\ before\ enhancement}{Exec.time\ after\ enhancement} = \frac{T(1-f)+Tf}{T(1-f)+\frac{Tf}{N}} = \frac{1}{(1-f)+\frac{f}{N}}$

The speedup is bound by the sequential fraction (1-f), i.e., by the part of the process that I cannot (or I'm not able) to enhance!

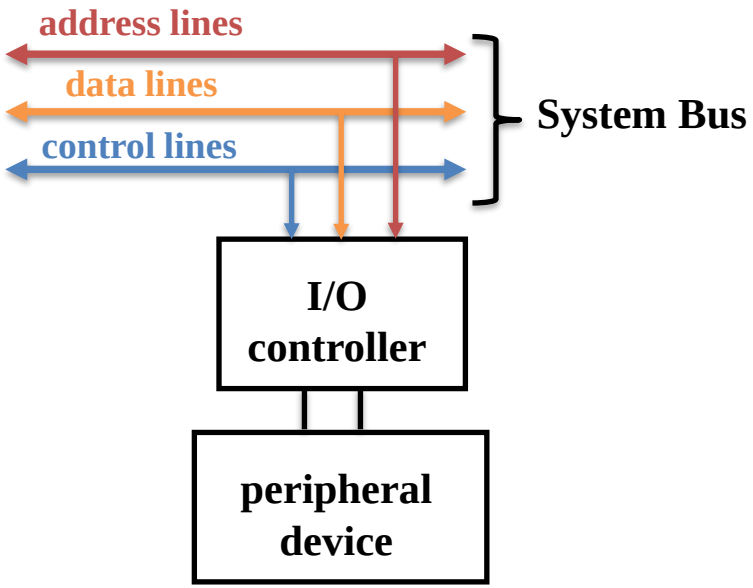
Not only performance

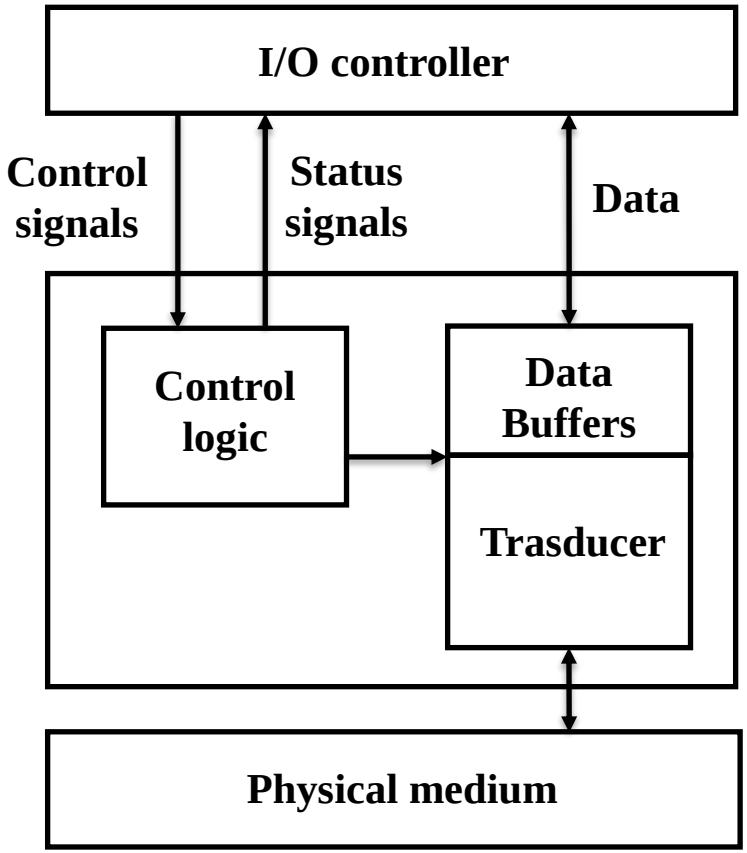
- Performance of the I/O subsystem is very important, but it is not all. Other important aspects are:
  - Reliability (affidabilità)
    - Mean Time To Failure (MTTF) metrics
    - Fault avoidance (better components)
    - Fault tolerance (introducing some level of redundancy)
  - Availability (disponibilità)
    - Mean Time To Repair (MTTR)
    - $Availability = \frac{MTTF}{MTTF+MTTR}$

I/O: control + data

- I/O devices have two ports
  - Control: both commands and status reports
    - How we tell the device what to do
    - How the device tells us about its features
    - How the device tells us about op status
  - Data
    - To/from device memory

Generic I/O device





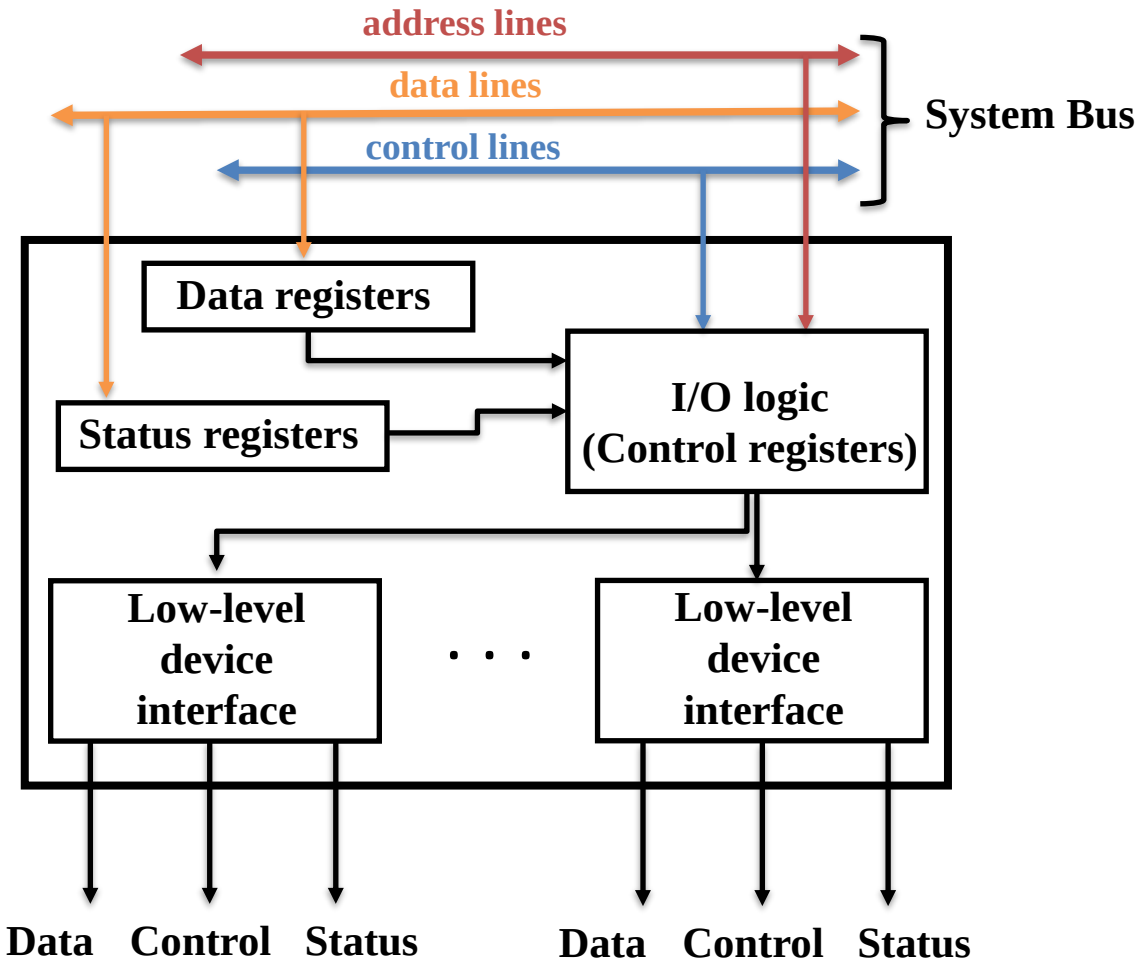
I/O controller functions

- Control and timing
- Processor communication
  - Command decoding (e.g., read sector X)
  - Data exchange
  - Status reporting
  - Address recognition
- Device communication (command, status info, data)
- Data buffering
  - To optimize data transfers
  - To compensate different speeds
- Error detection
  - Transmission errors (parity bis)
  - Electrical/mechanical errors

Logical I/O steps

- Control of the transfer from a device to processor:
  - CPU checks I/O module device status
  - I/O controller returns the status
  - If ready, CPU requests data transfer by means of a command to the controller
  - I/O controller gets data from the peripherical device
  - I/O controller transfers data to processor
- These steps require one or more bus arbitration actions to implement the ***communication protocol***

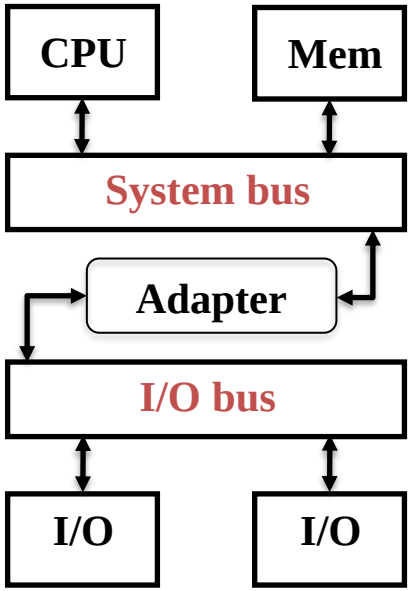
I/O controller diagram

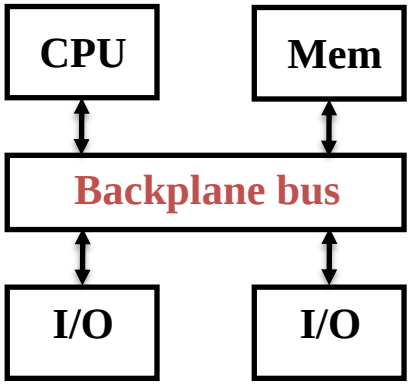


Bus interconnection

- A collection of data lines that is treated together as a single logical signal
  - Also used to indicate a shared collection of lines with multiple connected devices (called *taps*)
  - Performance factors: physical length, number of connected taps
- A bus is a *shared transmission medium*
  - Only one device at a time can successfully transmit
- The **system bus** connects major components (processor, memory, I/O bus)
  - Hundreds of separate lines
  - Lines classified in three groups: data, address, control

Examples of buses





- **System bus**

  Connects processor and memory, I/O interfaced with adapters
  - Short, few taps → Fast, high-bandwidth
  - Not standard (i.e., system specific)
- **I/O bus**

  Connects I/O devices, no direct connection with processor and memory
  - longer, more taps → slower, lower-bandwidth
  - Industry standard (e.g., ATA/SCSI)
- **Backplane bus**

  Connects CPU, memory, I/O devices
  - Long, many taps → slow, medium/low-bandwidth
  - Different devices with different bandwidths
  - Industry standard

Bus design

- Goals: high-performance, standardization, low cost
  - System bus emphasizes performance, I/O and Backplain buses emphasizes costs and standardization
- Design issues to address:
  - Are wires shared or separated?
    - Wider buses (i.e., higher bandwidth) are more expensive and more susceptible to skew
  - How is bus control acquired and released?
    - **Atomic transactions:** low utilization, low complexity.
    - **Split-transactions:** Request/replies can be interleaved; this means higher utilization but more complex design (e.g., IDs to identify different requests)
  - Is bus clocked?
  - How do we decide who gets the bus?

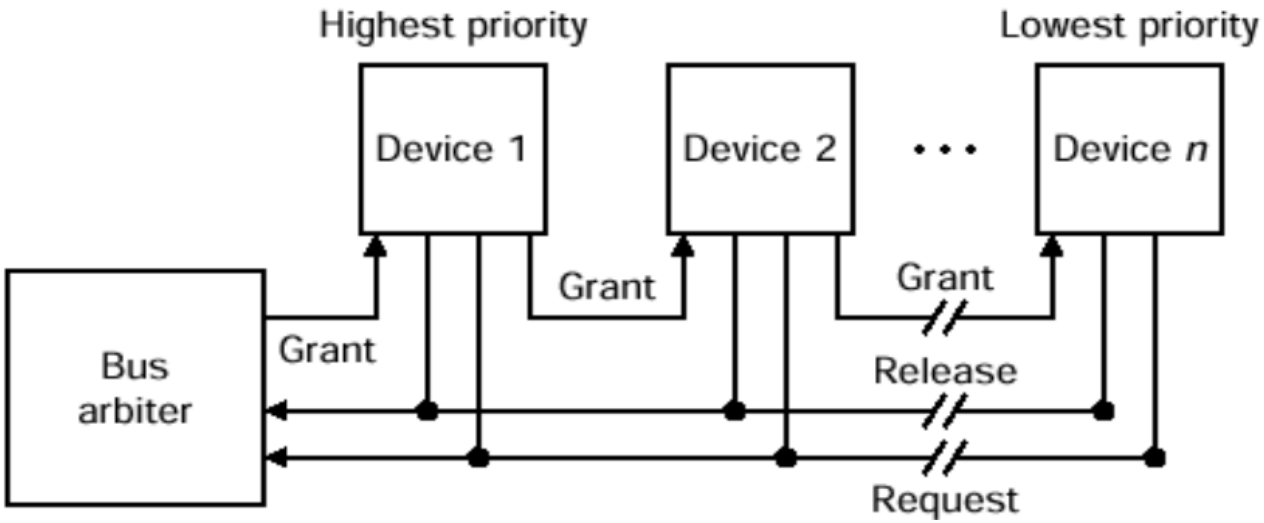
Bus clocking

- **Synchronous**
  - All devices connected to the bus share the same bus clock. Events happen at the edge of the clock signal
  - Possible protocol: at cycle X the I/O unit writes a READ request on the bus; at cycle X+Δ the unit can read the data from the bus. Δ is the maximum time to WRITE the data on the bus by a connected unit.
  - Potential clock skew issue
  - Limited to short buses (e.g., system bus)
- **Asynchronous**
  - The bus has no clock. No clock skew, it deals with devices with different speeds
  - Can be longer, thus slower. It requires ***handshaking*** protocols.
  - Possible protocol (3 control lines, 1 data line where data and address are multiplexed)
    1. UIO1 writes a READ (WRITE) request **ReadReq** (**WriteReq**) in the *control* line and the address in the *data* line
    2. UIO2 reads the address and writes an **ACK** into the control line to UIO1
    3. UIO2 writes the data on the bus and writes the **DataReady** control line to notify UIO1
    4. UIO1 reads the data and sends an **ACK** into the control line to UIO2

Bus arbitration

- Bus communications must be managed by a *communication protocol*
- **Bus master** concept: *unit that can initiate a bus request*
  - Simplest configuration: only one master
  - Buses typically have several masters
- **Arbitration** concept: choosing a master among multiple requests trying to implement priority and fairness (preventing starvation)
  - Only one master at a time (all others listen to the bus)
  - The role of the arbiter is to manage the bus requests and assign grants considering priorities and fairness.
- Implementation models for arbitration:
  - *Centralized*: a dedicated device (Arbiter) collects requests and then decide (potential *bottleneck problem*)
    - *Daisy-chain* (next slide), not fair
    - With independent request/reply lines (widely used)
  - *Distributed*: every device sees the requests simultaneously and participates to the selection of the next master (more complex to realize and it needs a lot of control lines)

Daisy-chain arbiter



- Devices connect to the bus in priority order
- High-priority devices intercept/deny requests by low-priority ones
- Simple to implement but slow and cannot ensure fairness
  - Request line: this is a wired-OR line.
  - Grant line: the signal is propagated through all the devices. If a device made a request, it will take control of the bus when it receives the grant signal and then negates the grant line for the next device.

bus examples

Characteristic	Firewire (1394)	USB 2.0	PCI Express	Serial ATA	Serial Attached SCSI
Intended use	External	External	Internal	Internal	External
Devices per channel	63	127	1	1	4
Basic data width (signals)	4	2	2 per lane	4	4
Theoretical peak bandwidth	50 MB/sec (Firewire 400) or 100 MB/sec (Firewire 800)	0.2 MB/sec (low speed), 1.5 MB/sec (full speed), or 60 MB/sec (high speed)	250 MB/sec per lane (1x); PCIe cards come as 1x, 2x, 4x, 8x, 16x, or 32x	300 MB/sec	300 MB/sec
Hot pluggable	Yes	Yes	Depends on form factor	Yes	Yes
Maximum bus length (copper wire)	4.5 meters	5 meters	0.5 meters	1 meter	8 meters
Standard name	IEEE 1394, 1394b	USB Implementors Forum	PCI-SIG	SATA-IO	T10 committee

I/O management

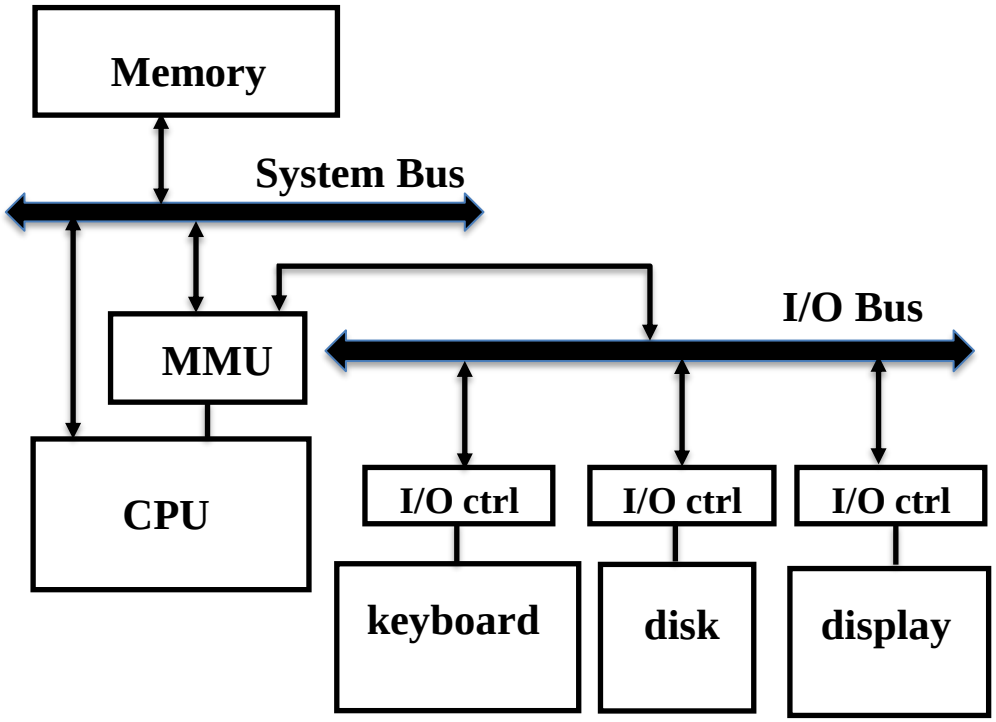
- How does I/O actually happen?
- We have to answer the following questions:
  1. **How does CPU give commands to I/O devices?**
  2. **How does CPU know when I/O devices completed operations?**
  3. **How do I/O devices execute data transfers?**

Sending commands to I/O devices

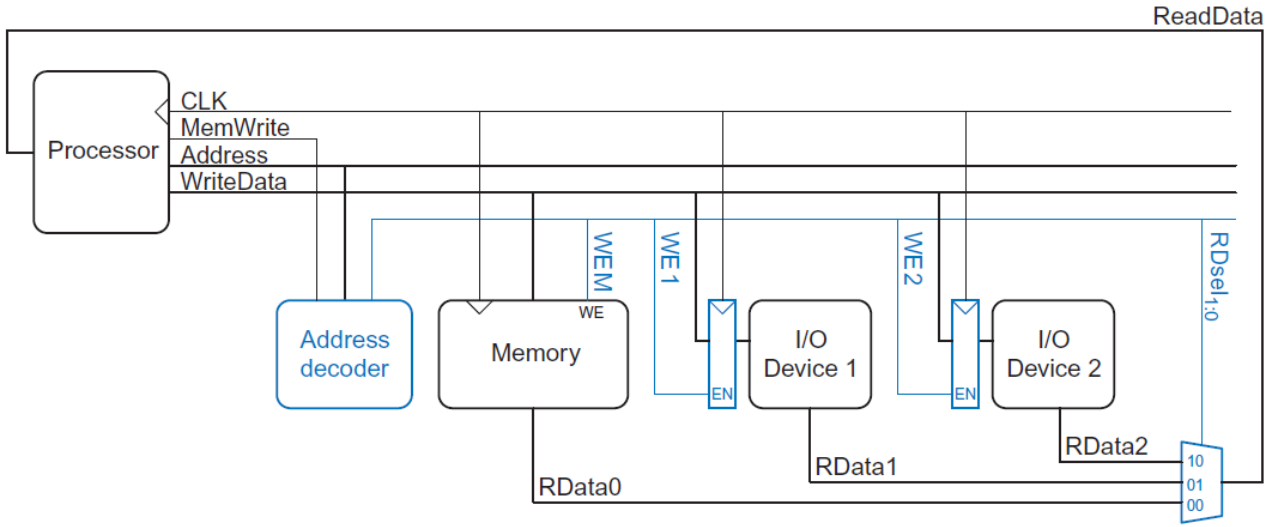
- **How does CPU give commands to I/O devices?**
- Who can send commands to I/O devices? **Only the OS!**
- How do programs in user space send I/O commands?
  - **By using System Calls!** They demand the task to the OS
- Two options for sending commands to I/O devices:
  - **I/O instructions**
    - ISA instructions that address the registers of the I/O device
    - *Privileged instructions* available only in kernel mode
    - Example architectures: Intel IA-32, IBM370
  - **Memory-mapped I/O**
    - A portion of the ***physical address space*** is reserved for I/O

Memory-mapped I/O

- The internal registers of the devices are mapped to main memory locations (**at *reserved physical addresses***)
- I/O commands are standard memory read/write
- Operations to those locations are redirected to the device controllers by the MMU
- User mode accesses to memory-mapped areas generate *memory violation* exceptions



Memory-mapped I/O



Example e9.1 COMMUNICATING WITH I/O DEVICES

Suppose I/O Device 1 in Figure e9.1 is assigned the memory address 0x20001000. Show the ARM assembly code for writing the value 7 to I/O Device 1 and for reading the output value from I/O Device 1.

Solution: The following assembly code writes the value 7 to I/O Device 1.

```
      MOV R1, #7
      LDR R2, =ioadr
      STR R1, [R2]
ioadr DCD 0x20001000
```

The address decoder asserts *WE1* because the address is 0x20001000 and *MemWrite* is TRUE. The value on the *WriteData* bus, 7, is written into the register connected to the input pins of I/O Device 1.

To read from I/O Device 1, the processor executes the following assembly code.

```
LDR R1, [R2]
```

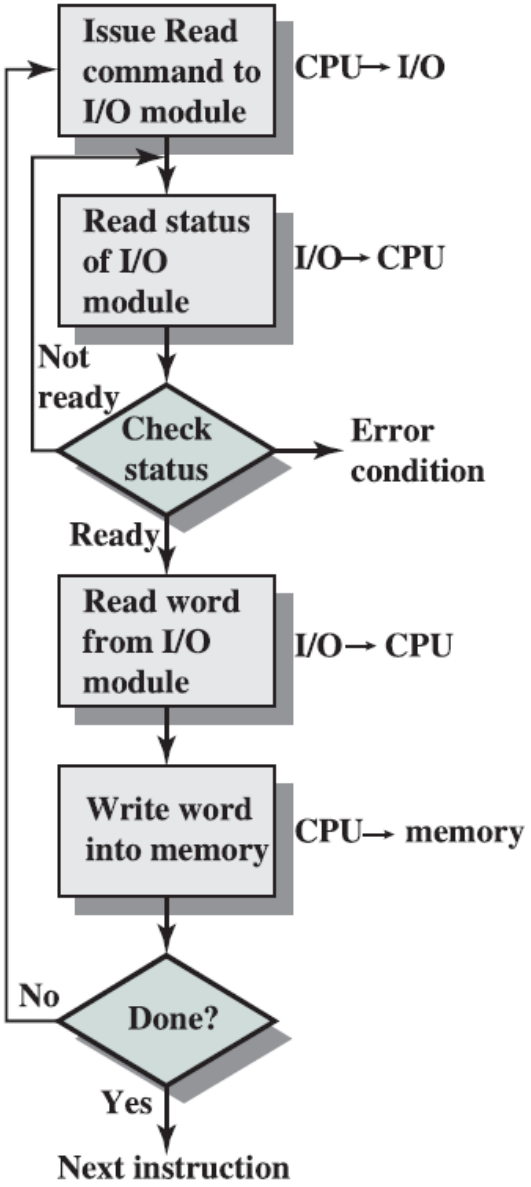
The address decoder sets $RDsel_{1:0}$ to 01, because it detects the address 0x20001000 and *MemWrite* is FALSE. The output of I/O Device 1 passes through the multiplexer onto the *ReadData* bus and is loaded into R1 in the processor.

Querying I/O status

- **How does the CPU know if the operation request has completed?**
  - **Programmed I/O**
    - CPU directly control I/O operation and status
  - **Interrupt-driven I/O**
    - An ***interrupt*** is an *asynchronous event coming from a device* (e.g., I/O module, timer, DMA controller)
    - Sometimes ***exceptions*** are also called interrupts (or comprise interrupts). In general exceptions are *synchronous events* that disrupt program execution (e.g., arithmetic overflow, undefined instruction).

Programmed I/O

- CPU has direct control over I/O
  - Sensing status
  - Read/write commands
  - Data transferring
- The I/O device has a *passive* role
- CPU waits for I/O module to complete operations (***polling***)



Programmed I/O

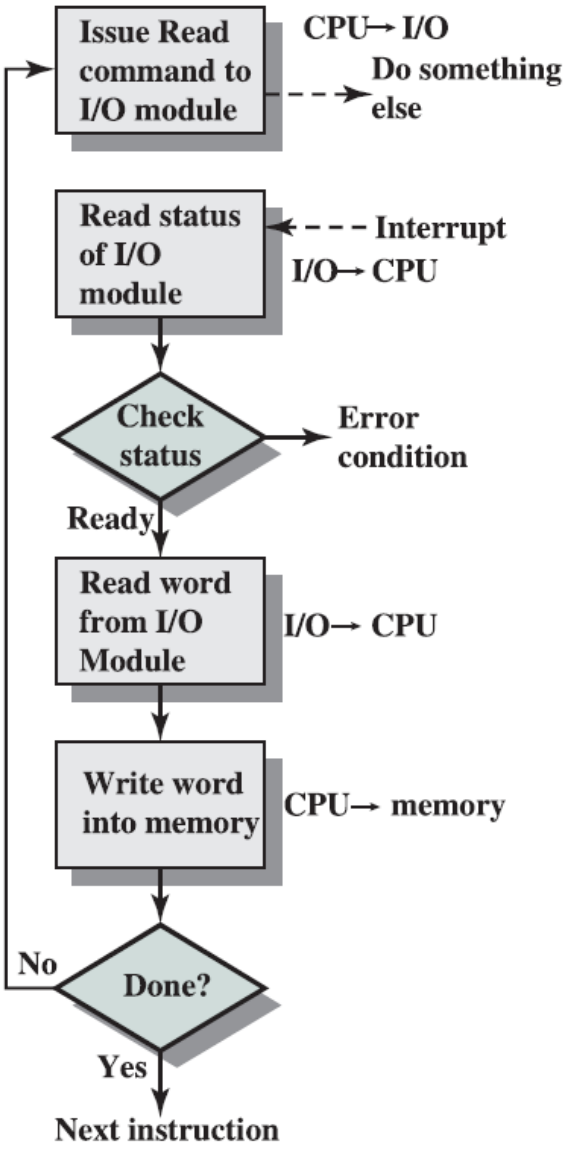
- Polling
  - *Processor queries I/O device status register*

    READ example:
    1. write I/O control register for READ op
    2. while (ready-bit == 0) do; // ***busy waiting***
    3. *load the data in memory*

  - *Pros: Simple to implement*
  - *Cons:* ***Waste of processor's cycles***
    - *The CPU is much faster than any I/O device*
    - *The CPU may read the Status register many times only to find that the device has not completed the operation*

Interrupt-driven I/O

- The problem of programmed I/O is CPU waste of time
- **Interrupts**: alternative to polling
  - The CPU issues commands to a device and continues doing other tasks
  - I/O device generates an interrupt when the status changes, i.e., the data is ready
  - I/O interrupts are asynchronous
  - I/O interrupts are prioritized
    - High-bandwidth I/O devices have higher priority than low-bandwidth ones



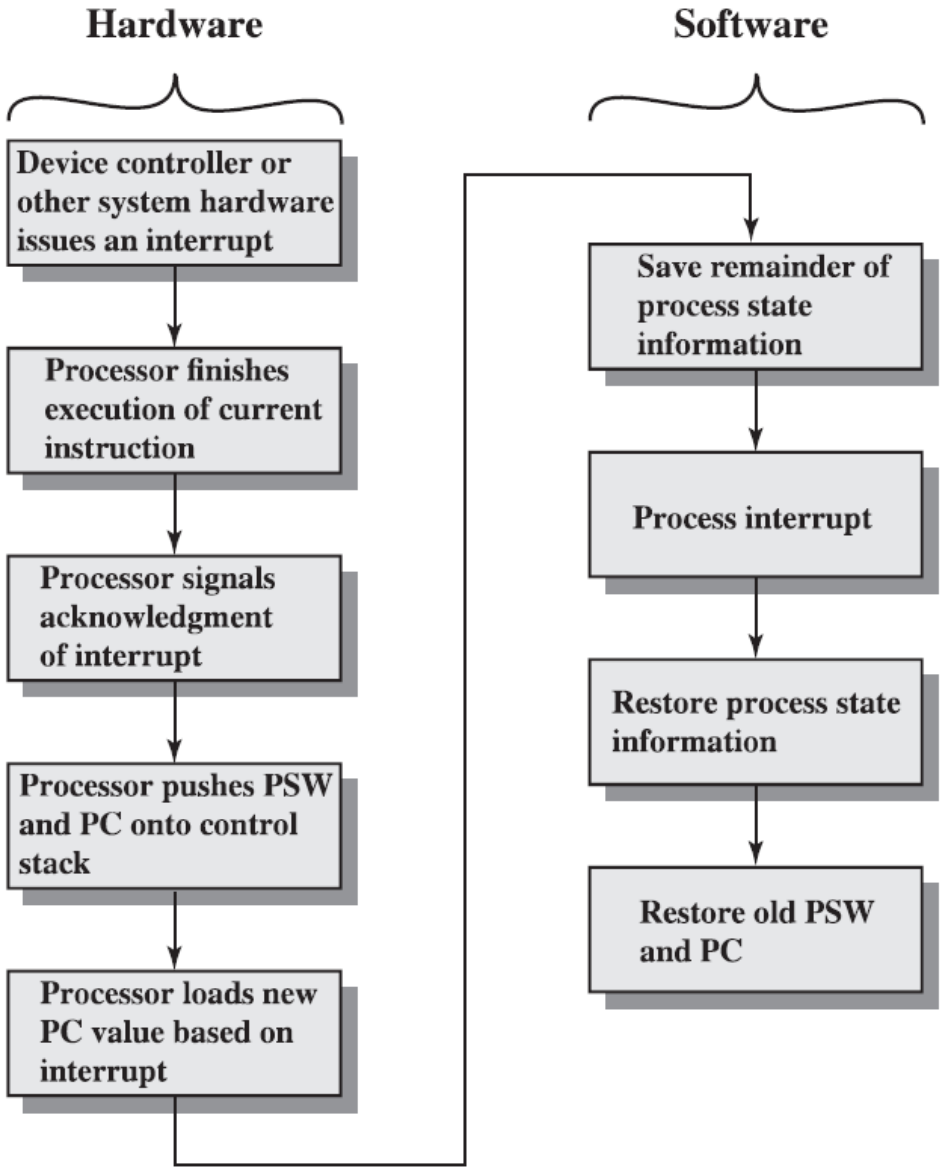
Interrupt-driven I/O

- CPU viewpoint
  - Issue I/O command (e.g., a memory-mapped load)
  - Do other work
  - Check for interrupt at the end of the fetch-execute cycle
  - If interrupt received
    - Save current context (i.e., registers, not necessarily all) and switch to *privileged level* (i.e., PL1 on ARM processor)
    - On the basis of the interrupt ID and interrupt priority the associated interrupt handler is ready to be executed
      - **NOTE**: The previous steps are atomic from the OS viewpoint!
    - Execute the handler (i.e., the OS handles the interrupt)
    - Restore context for executing the next instruction
      - **NOTE**: the restore is an atomic operation for the OS!

Interrupt management steps

PSW is the Program Status Word register, the equivalent of the **CPSR/APSR** register for ARM processors

```
while(true) {
   try {
      IR = fetch(PC);
      decode(IR);
      execute(IR);
      update(PC);
   } catch (exception e) {
      exception_management();
   }
   if(interrupt) {
      interrupt_management();
   }
}
```



Polling overhead

- **Assumptions**: 500MHz CPU, polling cost 400 cycles
- **MOUSE**: slow-bandwidth device
  - we want to poll 30 times per second
  - Cycles per second for polling: (30 poll/s) * 400 = 12000 cycles/s
  - Percentage of cycles spent for polling: 12K/500M = **0.002%**
  - **Acceptable overhead!**
- **DISK**: high-bandwidth device (4MB/s transfer rate, 16B interface)
  - To not miss data, we must poll often enough: (4MB/s)/16B = 250 K/s
  - Cycles per second for polling: 250 K/s * 400 (cycles/s) = 100 M cycles/s
  - Percentage of cycles spent for polling: 100 M/500M = **20%**
  - **Not acceptable!!** We spent 20% of cycles just for checking the I/O registers, not for data transfer

Interrupt overhead

- **Assumptions**: 500MHz CPU, interrupt handling takes 400 cycles

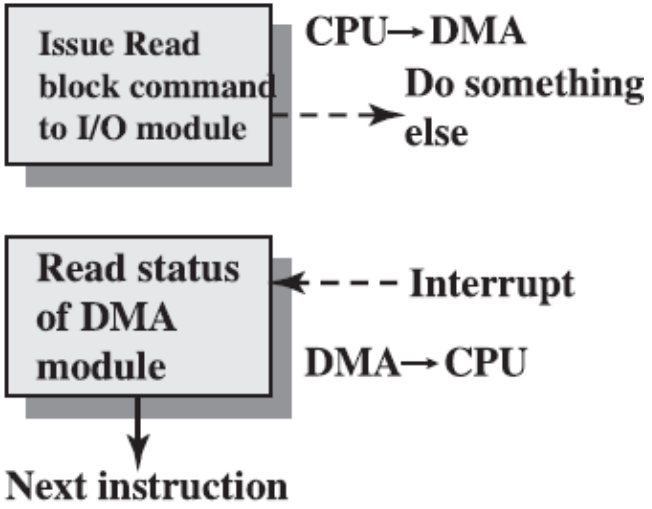
- **DISK** (4MB/s transfer rate, 16B interface)
  - The processor is involved for each data transfer (16 B)
  - (4M B/s) / (16 B) * [400 cycles / 500M cycles/s] = **20 %**
  - Same overhead of polling *if the disk is fully utilized*, but in this case **the CPU can execute other instructions** (overlapping the I/O operation with other operations)
  - In CPU-bound applications, the I/O devices are used for a fraction of time!
- Let's suppose that the data transfer time takes 100 cycles
  - The percentage of time the processor spends transferring data is (4M B/s) / (16 B) * [100 cycles / 500M cycles/s] = **5 %**

Data transfers (DMA)

- **How do I/O devices execute data transfers?**
- Interrupt-driven I/O remove the problems of polling
- Still the OS must transfer data one word at a time. This is ok only for low-bandwidth I/O devices (e.g., mouse)
  - **NOTE**: n general, the cost of managing interrupt is higher than polling
- **Direct Memory Access (DMA)**
  - A mechanisms that provides an I/O device controller with the ability to transfer data directly to and from main memory *without involving the CPU*
  - DMA decouples the CPU-Memory protocol from the I/O device-memory protocol
  - Interrupts used by the I/O device to communicate with the CPU only on completion of the I/O transfer or when an error occurs.

DMA controller

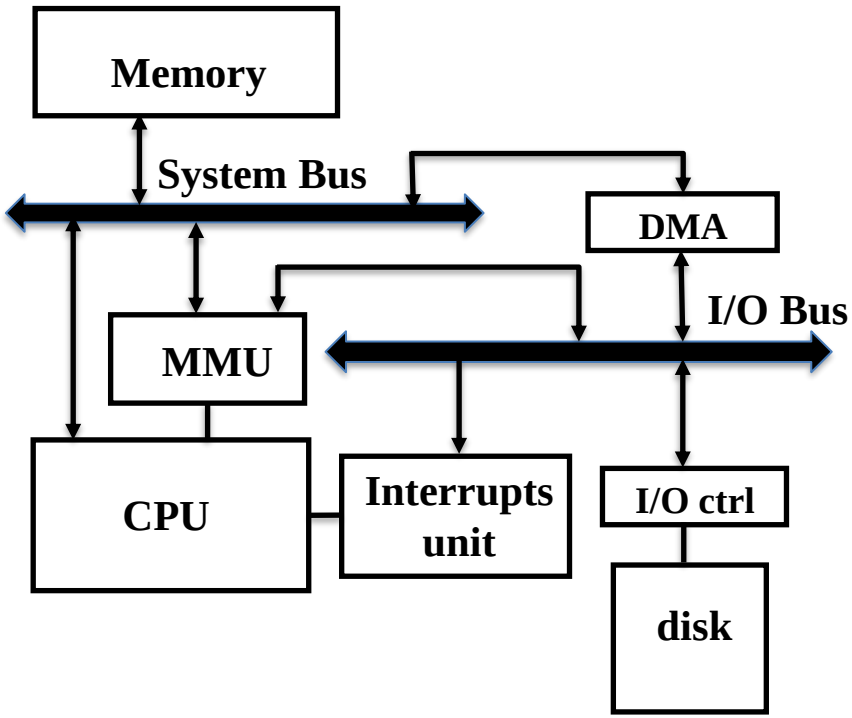
- *DMA is implemented with a specialized controller*
- To do DMA, an I/O device is attached to a DMA controller
  - Multiple devices can be connected to the same controller
- The controller itself is seen as a memory-mapped I/O device
- The DMA controller takes care of bus arbitration and data transfer
  - It becomes the bus master
- The DMA controller and the CPU contend the memory bus



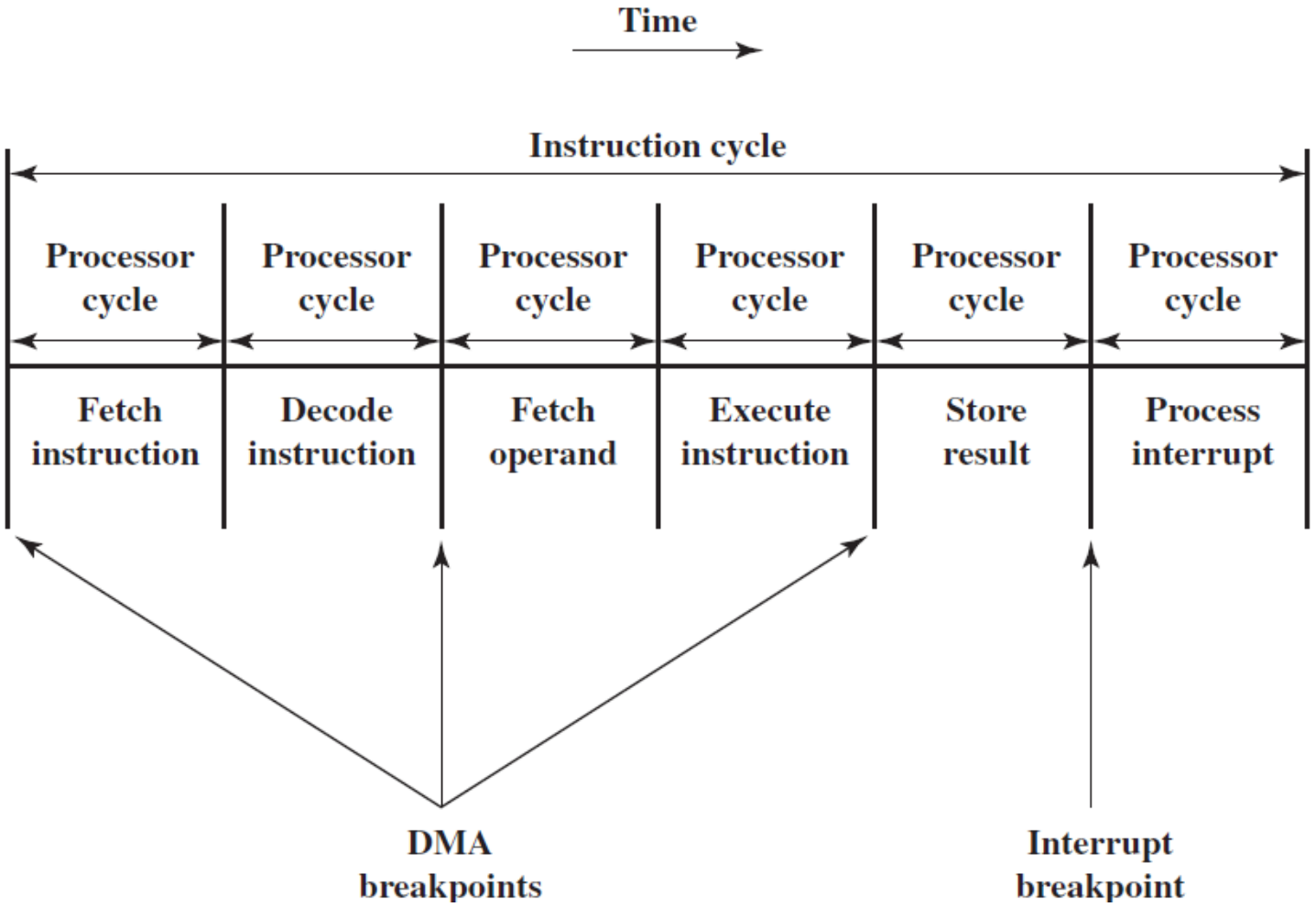
DMA transfer

- There are three steps in a DMA transfer:

1. CPU sets up DMA providing *identity*, *op*, *memory address*, and number of *bytes to transfer*
2. DMA starts the op on the device and arbitrates for the connection. It transfers the data from the device or memory
3. Once the DMA transfer is complete, the controller sends an interrupt to the CPU



DMA and Interrupt Breakpoints



DMA overhead

- **Assumptions**:
  - 500MHz CPU, disk device with 4MB/s transfer rate, 16B interface, 50% utilization
  - Interrupt handling takes 400 cycles
  - Data transfer takes 100 cycles
  - DMA setup takes 1600 cycles, it transfers a 16KB block at a time
- Processor overhead for **interrupt-driven I/O without DMA**
  - The processor is involved for each data transfer (16 B)
  - 0.5 * (4 MB/s) / (16 B) * [(400+100) cycles / 500M cycles/s] = **12.5 %**
- Processor overhead for **interrupt-driven I/O with DMA transfer**
  - The processor is involved once per block transfer (16 KB)
  - 0.5 * (4M B/s)/ (16KB) * [(1600+ 400) / (500 M cycles/s)] = **0.05 %**

DMA and Memory Hierarchy

- **Without DMA**: processor initiates all data transfers
  - All transfers go through address translation (MMU)
  - Transfers can be of any size and cross virtual page boundaries
  - No impact on the cache hierarchy, caches never contain stale data
- **With DMA:** the DMA controller initiates data transfers
  - There is another path to the memory system!
  - Potential issues:
    1. Do DMA controllers use ***virtual or physical addresses***?
    2. What if they write data to a **cached memory location**?

Virtual vs Physical DMA

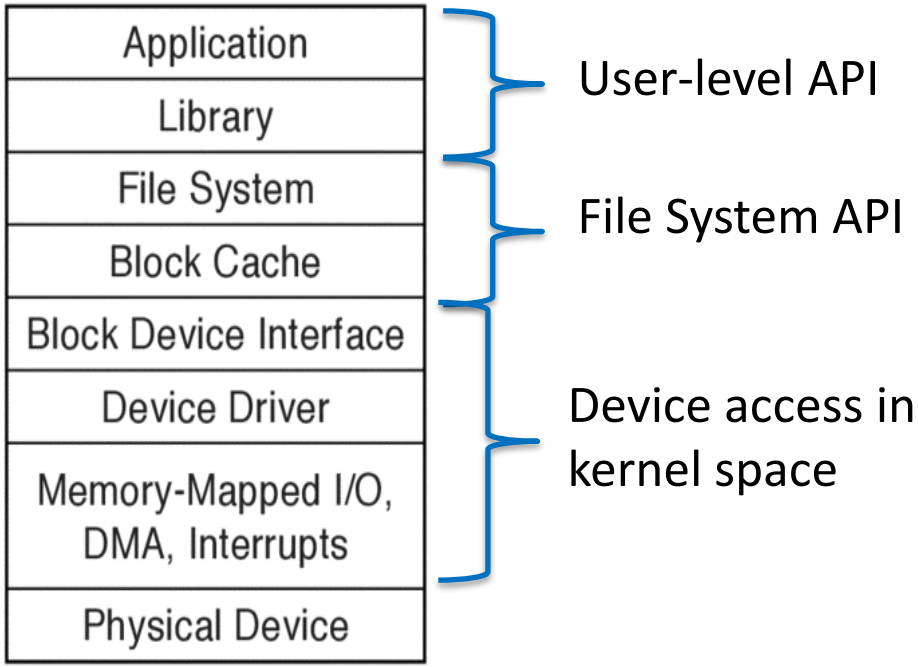
- Which addresses does processor specify to DMA controller?
- **Virtual DMA**:
  - The DMA controller must do address translation internally by using a small cache (TLB) initialized by the OS when it requests an I/O transfer
  - Complex but flexible (e.g., large cross-page transfers)
- **Physical DMA:**
  - The DMA controller works with physical addresses
  - Transfers at page granularity, the OS breaks large transfers into page-size chunks
  - More simple, but less flexible

DMA and caching

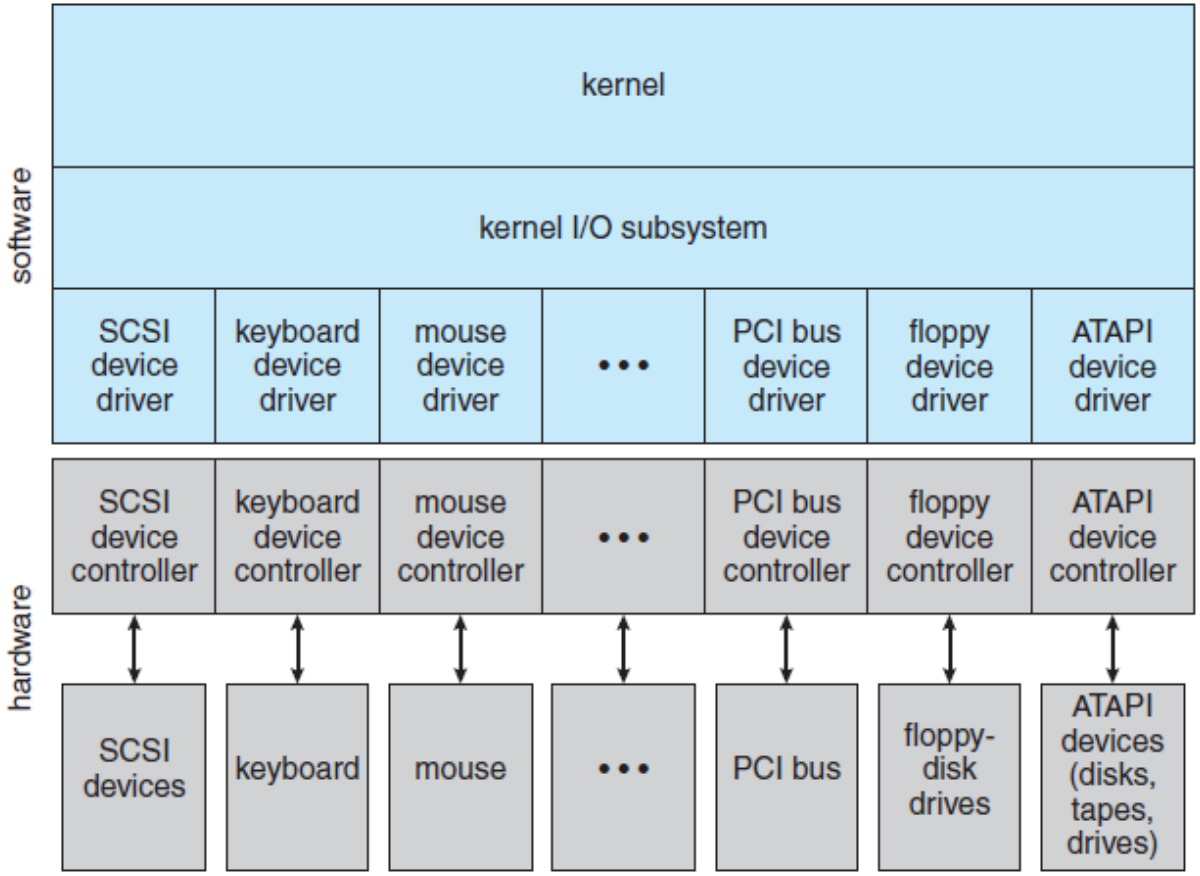
- Caching reduces CPU's instruction and data access latency, and reduces CPU's use of memory, thus leaving more memory/bus bandwidth for DMA transfers
- But caches introduce a **coherence problem** for DMA
  - If the DMA writes into main memory, then cached version of data is stale
  - For Write-Back caches, the DMA controller might read old data values from memory while cached data have the dirty-bit set
- The solution is the selective *flushing*/*invalidations* of cached lines involved in DMA transfers
  - Requires extra logic for HW cache coherence!

Device Access

- A software stack that provides ways to access a wide range of I/O devices
- A common interface
  - In POSIX equivalent to file access
  - In terms of open/close, read/write system calls
- Device drivers for each specific device
  - Upper part HW-**independent (to enhance portability)**
  - Lower part HW-**dependent**



Kernel I/O structure

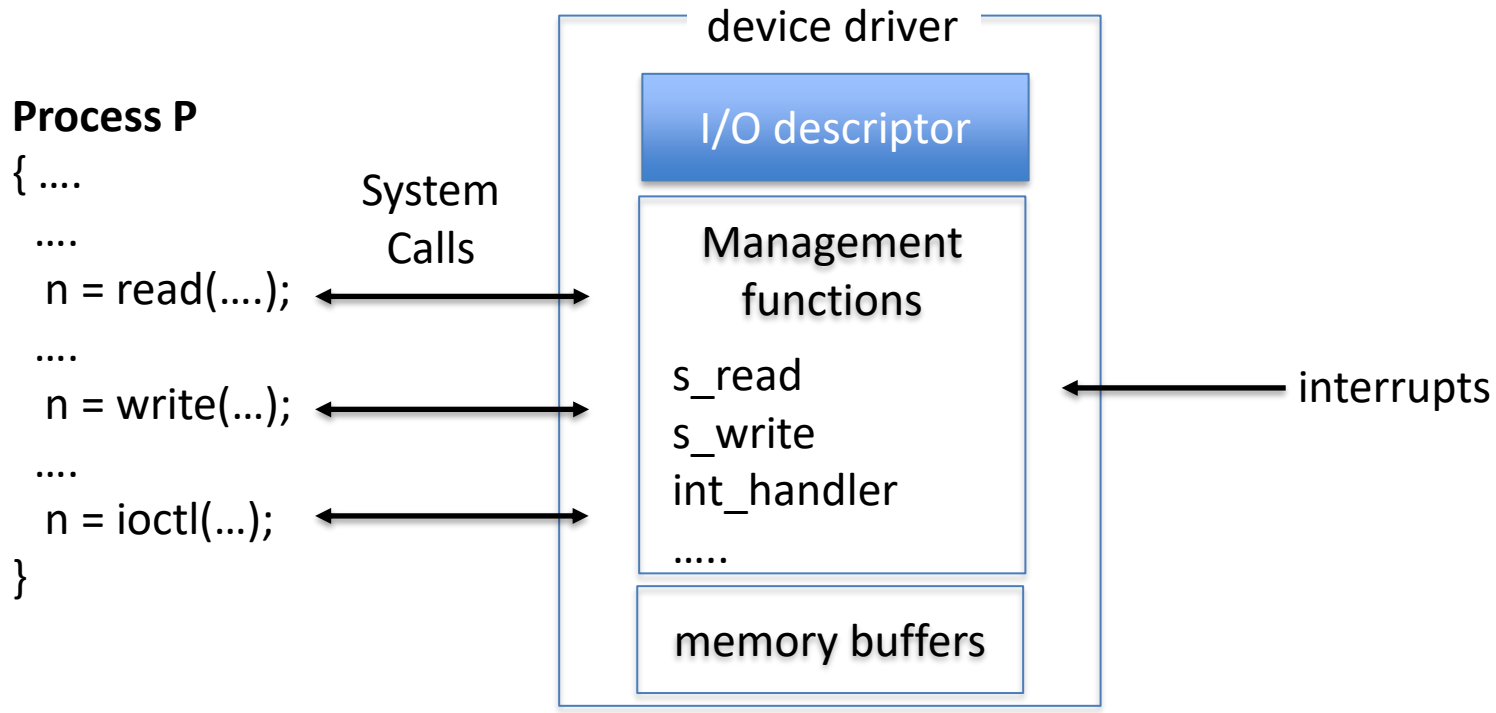


- The purpose of the device driver layer is to hide the differences among device controllers

Device driver

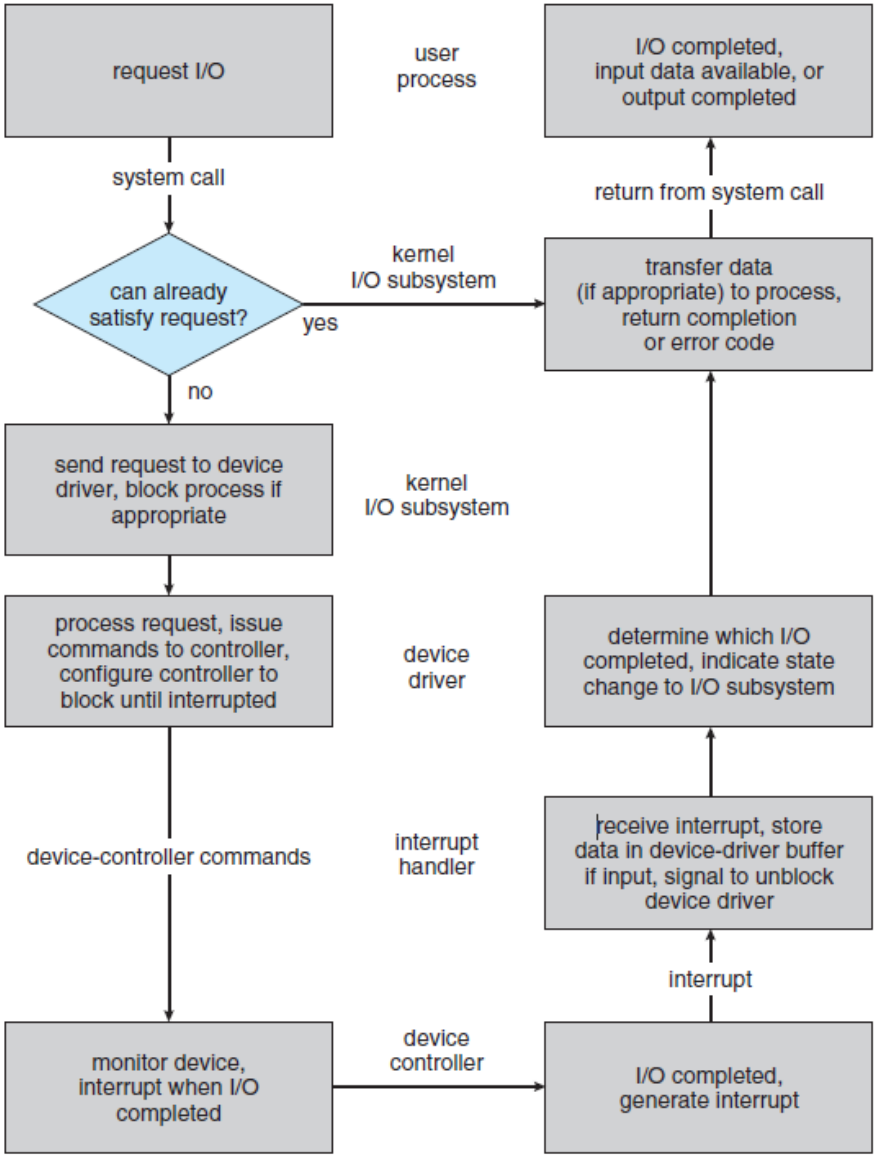
- The file system:
  - Defines a name space for devices and their identifiers (i.e., <major, minor> numbers)
  - Implements caching policies
  - It is HW-independent
- The ***device driver***:
  - It works in kernel space
  - Interfaces with the device controller (HW-dependent)
  - Manages synchronization with the device
  - Manages data transfer to/from device (by interfacing the DMA controller) and to/from user processes
  - Manages device failures

Device driver

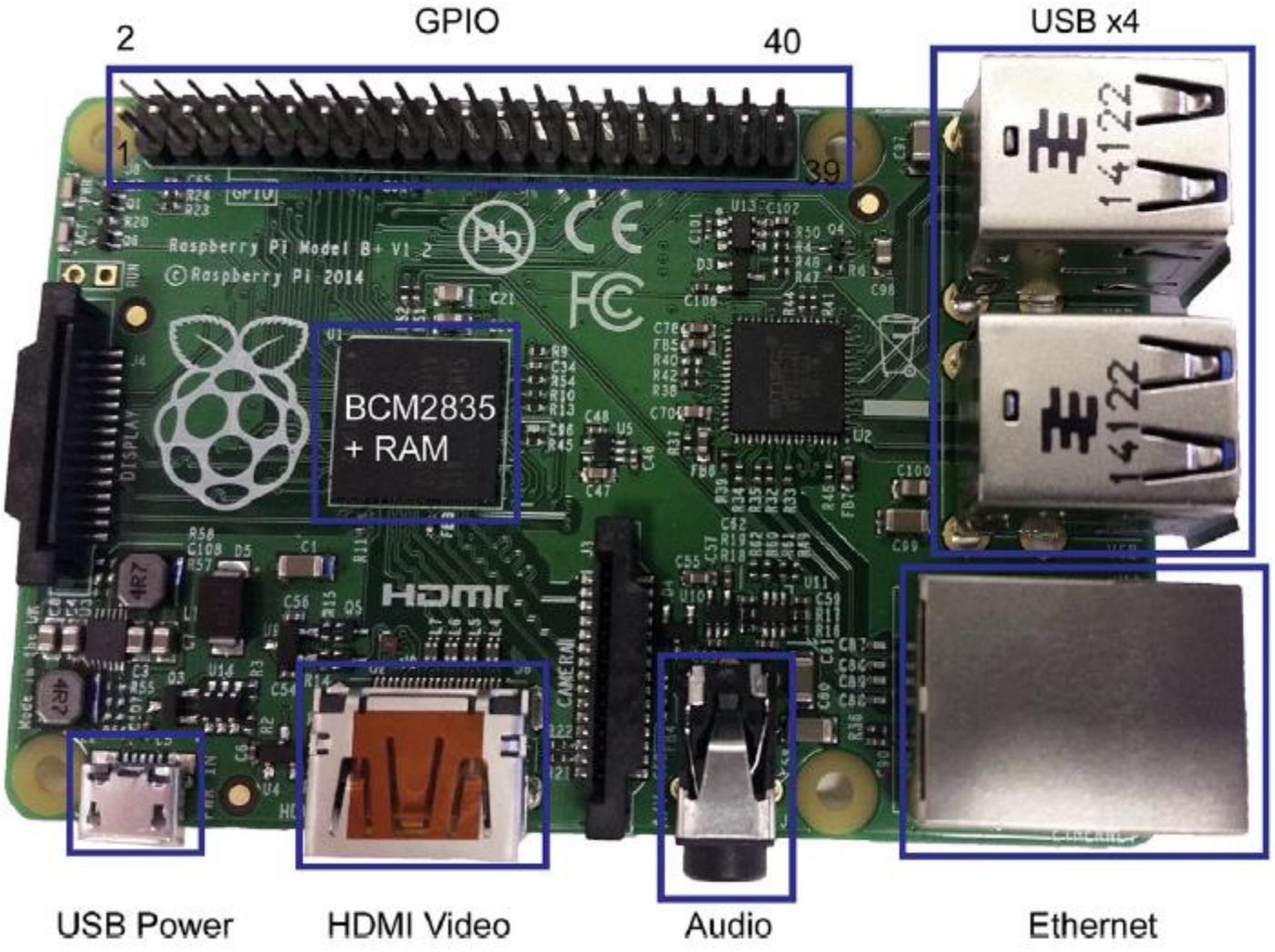


- I/O descriptor contains all information to interact (i.e., communicate and synchronize) with the specific device
  - Buffer pointers, counters, synchronization variables, status, etc.

I/O request life cycle



ARM example



Raspbery Pi Model B+

MM I/O on BCM2835

- The memory-mapped I/O on the BCM2835 is found at ***physical addresses*** 0x20000000-0x20FFFFFF
- Loads/stores refers to **virtual addresses**
- To access memory-mapped I/O memory regions it is *necessary to map the physical addresses to the program's virtual address space.*
  - *mmap system call*

```
#include <sys/mman.h>
#define BCM2835_PERI_BASE 0x20000000
#define GPIO_BASE         (BCM2835_PERI_BASE + 0x200000)
volatile unsigned int    *gpio; //Pointer to base of gpio
#define GPLEV0            (* (volatile unsigned int *) (gpio + 13))
#define BLOCK_SIZE        (4*1024)
void pioInit(){
  int mem_fd;
  void *reg_map;

  // /dev/mem is a psuedo-driver for accessing memory in Linux
  mem_fd = open("/dev/mem", O_RDWR|O_SYNC);
  reg_map = mmap(
   NULL,                  // Address at which to start local mapping (null = don't-care)
   BLOCK_SIZE,            // 4KB mapped memory block
   PROT_READ|PROT_WRITE,  // Enable both reading and writing to the mapped memory
   MAP_SHARED,            // Nonexclusive access to this memory
   mem_fd,                // Map to /dev/mem
   GPIO_BASE);            // Offset to GPIO peripheral

  gpio = (volatile unsigned *)reg_map;
  close(mem_fd);
}
```